

Note: Qwen3.5-4b available in huggingface format. That need to converted in rkllm format.

Refer link:

https://docs.google.com/document/d/1qh0VC1hfRVcoSGxLvwc8R9XxsJRxbpLCiMR6i9Asvo/edit?usp=drive_link

Environment:

Hardware SBC: Orange pi pro 5(16GB)

OS: ubuntu-24.04-preinstalled-server-arm64-orangepi-5-pro.img.xz

Steps:

1. Connect OrangePiPro with ssh
2. Create python virtual environment and activate
3. Git clone <https://github.com/airockchip/rknn-llm>, or simply download and copy https://github.com/airockchip/rknn-llm/blob/main/rkllm-runtime/Linux/librkllm_api/aarch64/librkllmrt.so at /usr/local/
4. Run
\$> ldconfig
5. Copy qwen3.5-4b model in rkllm format which is discussed in the reference document above.
6. Open parallel ssh connection and run to monitor npu load

\$> watch -n 1 sudo cat /sys/kernel/debug/rknpu/load

7. Run below sample code keeping in "some_filename.py" as "yourenv/bin/python some_filename.py"

Sample Test code <run>:

```
# CMD="python3 flask_server.py --rkllm_model_path $MODEL_PATH
--target_platform $TARGET_PLATFORM --lora_model_path $LORA_PATH
--prompt_cache_path $PROMPT_FILE_PATH"
import ctypes
import sys
import os
import subprocess
import resource
import threading
import time
import argparse
import json
import uuid
import signal
```

```

# Set the dynamic library path
rkllm_lib = ctypes.CDLL('/usr/lib/librkllmrt.so')
# Define the structures from the library
RKLLM_Handle_t = ctypes.c_void_p
userdata = ctypes.c_void_p(None)

LLMCallState = ctypes.c_int
LLMCallState.RKLLM_RUN_NORMAL = 0
LLMCallState.RKLLM_RUN_WAITING = 1
LLMCallState.RKLLM_RUN_FINISH = 2
LLMCallState.RKLLM_RUN_ERROR = 3

RKLLMInputType = ctypes.c_int
RKLLMInputType.RKLLM_INPUT_PROMPT = 0
RKLLMInputType.RKLLM_INPUT_TOKEN = 1
RKLLMInputType.RKLLM_INPUT_EMBED = 2
RKLLMInputType.RKLLM_INPUT_MULTIMODAL = 3

RKLLMInferMode = ctypes.c_int
RKLLMInferMode.RKLLM_INFER_GENERATE = 0
RKLLMInferMode.RKLLM_INFER_GET_LAST_HIDDEN_LAYER = 1
RKLLMInferMode.RKLLM_INFER_GET_LOGITS = 2
class RKLLMExtendParam(ctypes.Structure):
    _fields_ = [
        ("base_domain_id", ctypes.c_int32),
        ("embed_flash", ctypes.c_int8),
        ("enabled_cpus_num", ctypes.c_int8),
        ("enabled_cpus_mask", ctypes.c_uint32),
        ("n_batch", ctypes.c_uint8),
        ("use_cross_attn", ctypes.c_int8),
        ("reserved", ctypes.c_uint8 * 104)
    ]

class RKLLMParam(ctypes.Structure):
    _fields_ = [
        ("model_path", ctypes.c_char_p),
        ("max_context_len", ctypes.c_int32),
        ("max_new_tokens", ctypes.c_int32),
        ("top_k", ctypes.c_int32),
        ("n_keep", ctypes.c_int32),
        ("top_p", ctypes.c_float),
        ("temperature", ctypes.c_float),
        ("repeat_penalty", ctypes.c_float),

```

```

        ("frequency_penalty", ctypes.c_float),
        ("presence_penalty", ctypes.c_float),
        ("mirostat", ctypes.c_int32),
        ("mirostat_tau", ctypes.c_float),
        ("mirostat_eta", ctypes.c_float),
        ("skip_special_token", ctypes.c_bool),
        ("ignore_eos_token", ctypes.c_bool),
        ("is_async", ctypes.c_bool),
        ("extend_param", RKLLMExtendParam),
    ]

class RKLLMLoraAdapter(ctypes.Structure):
    _fields_ = [
        ("lora_adapter_path", ctypes.c_char_p),
        ("lora_adapter_name", ctypes.c_char_p),
        ("scale", ctypes.c_float)
    ]

class RKLLMEmbedInput(ctypes.Structure):
    _fields_ = [
        ("embed", ctypes.POINTER(ctypes.c_float)),
        ("n_tokens", ctypes.c_size_t)
    ]

class RKLLMTokenInput(ctypes.Structure):
    _fields_ = [
        ("input_ids", ctypes.POINTER(ctypes.c_int32)),
        ("n_tokens", ctypes.c_size_t)
    ]

class RKLLMImageInput(ctypes.Structure):
    _fields_ = [
        ("image_embed", ctypes.POINTER(ctypes.c_float)),
        ("n_image_tokens", ctypes.c_size_t),
        ("n_image", ctypes.c_size_t),
        ("image_start", ctypes.c_char_p),
        ("image_end", ctypes.c_char_p),
        ("image_content", ctypes.c_char_p),
        ("image_width", ctypes.c_size_t),
        ("image_height", ctypes.c_size_t),
    ]

class RKLLMVideoInput(ctypes.Structure):

```

```

    _fields_ = [
        ("video_embed", ctypes.POINTER(ctypes.c_float)),
        ("n_frame_tokens", ctypes.c_size_t),
        ("n_frame_per_video", ctypes.c_size_t),
        ("n_video", ctypes.c_size_t),
        ("video_start", ctypes.c_char_p),
        ("video_end", ctypes.c_char_p),
        ("video_content", ctypes.c_char_p),
        ("frame_width", ctypes.c_size_t),
        ("frame_height", ctypes.c_size_t),
    ]

```

```

class RKLLMMultiModalInput(ctypes.Structure):
    _fields_ = [
        ("prompt", ctypes.c_char_p),
        ("image", RKLLMImageInput),
        ("video", RKLLMVideoInput),
    ]

```

```

class RKLLMInputUnion(ctypes.Union):
    _fields_ = [
        ("prompt_input", ctypes.c_char_p),
        ("embed_input", RKLLMEmbedInput),
        ("token_input", RKLLMTokenInput),
        ("multimodal_input", RKLLMMultiModalInput)
    ]

```

```

class RKLLMInput(ctypes.Structure):
    _anonymous_ = ("input_data",)
    _fields_ = [
        ("role", ctypes.c_char_p),
        ("enable_thinking", ctypes.c_bool),
        ("input_type", RKLLMInputType),
        ("input_data", RKLLMInputUnion)
    ]

```

```

class RKLLMLoraParam(ctypes.Structure):
    _fields_ = [
        ("lora_adapter_name", ctypes.c_char_p)
    ]

```

```

class RKLLMPromptCacheParam(ctypes.Structure):
    _fields_ = [

```

```

        ("save_prompt_cache", ctypes.c_int),
        ("prompt_cache_path", ctypes.c_char_p)
    ]

class RKLLMSamplingParam(ctypes.Structure):
    _fields_ = [
        ("top_k", ctypes.c_int32),
        ("top_p", ctypes.c_float),
        ("temperature", ctypes.c_float),
        ("repeat_penalty", ctypes.c_float),
        ("frequency_penalty", ctypes.c_float),
        ("presence_penalty", ctypes.c_float),
        ("mirostat", ctypes.c_int32),
        ("mirostat_tau", ctypes.c_float),
        ("mirostat_eta", ctypes.c_float),
    ]

class RKLLMInferParam(ctypes.Structure):
    _fields_ = [
        ("mode", RKLLMInferMode),
        ("lora_params", ctypes.POINTER(RKLLMLoraParam)),
        ("prompt_cache_params", ctypes.POINTER(RKLLMPromptCacheParam)),
        ("sampling_params", ctypes.POINTER(RKLLMSamplingParam)),
        ("keep_history", ctypes.c_int),
        ("max_new_tokens", ctypes.c_int32),
    ]

class RKLLMResultLastHiddenLayer(ctypes.Structure):
    _fields_ = [
        ("hidden_states", ctypes.POINTER(ctypes.c_float)),
        ("embd_size", ctypes.c_int),
        ("num_tokens", ctypes.c_int)
    ]

class RKLLMResultLogits(ctypes.Structure):
    _fields_ = [
        ("logits", ctypes.POINTER(ctypes.c_float)),
        ("vocab_size", ctypes.c_int),
        ("num_tokens", ctypes.c_int)
    ]

class RKLLMPerfStat(ctypes.Structure):
    _fields_ = [

```

```

        ("prefill_time_ms", ctypes.c_float),
        ("prefill_tokens", ctypes.c_int),
        ("generate_time_ms", ctypes.c_float),
        ("generate_tokens", ctypes.c_int),
        ("memory_usage_mb", ctypes.c_float)
    ]

class RKLLMResult(ctypes.Structure):
    _fields_ = [
        ("text", ctypes.c_char_p),
        ("token_id", ctypes.c_int),
        ("last_hidden_layer", RKLLMResultLastHiddenLayer),
        ("logits", RKLLMResultLogits),
        ("perf", RKLLMPerfStat)
    ]

# Define global variables to store the callback function output
system_prompt = ''
global_text = []
global_state = -1
split_byte_data = bytes(b'') # Used to store the segmented byte data

# Per-request context for function calling
tool_call_context = {"pending_tools": None, "tool_response_str":
"tool_response"}

# Define the callback function
def callback_impl(result, userdata, state):
    global global_text, global_state, split_byte_data
    if state == LLMCallState.RKLLM_RUN_FINISH:
        global_state = state
        sys.stdout.flush()
    elif state == LLMCallState.RKLLM_RUN_ERROR:
        global_state = state
        print("run error")
        sys.stdout.flush()
    elif state == LLMCallState.RKLLM_RUN_NORMAL:
        global_state = state
        global_text += result.contents.text.decode('utf-8')
    return 0

```

```

# Connect the callback function between the Python side and the C++
side
LLMResultCallback_type = ctypes.CFUNCTYPE(ctypes.c_int,
ctypes.POINTER(RKLLMResult), ctypes.c_void_p, ctypes.c_int)
LLMTokenizerCallback_type = ctypes.CFUNCTYPE(ctypes.c_int,
ctypes.c_void_p, ctypes.c_char_p, ctypes.c_int32,
ctypes.POINTER(ctypes.c_int32), ctypes.c_int32)
LLMGetEmbedCallback_type = ctypes.CFUNCTYPE(ctypes.c_int,
ctypes.c_void_p, ctypes.POINTER(ctypes.c_int32), ctypes.c_uint64,
ctypes.c_void_p, ctypes.c_uint64)

class RKLLMCallback(ctypes.Structure):
    _fields_ = [
        ("result_callback", LLMResultCallback_type),
        ("result_userdata", ctypes.c_void_p),
        ("tokenizer_callback", LLMTokenizerCallback_type),
        ("tokenizer_userdata", ctypes.c_void_p),
        ("embed_callback", LLMGetEmbedCallback_type),
        ("embed_userdata", ctypes.c_void_p),
    ]

callback = LLMResultCallback_type(callback_impl)

# Define the RKLLM class, which includes initialization, inference, and
release operations for the RKLLM model in the dynamic library
class RKLLM(object):
    def __init__(self, model_path, lora_model_path = None,
prompt_cache_path = None, platform = "rk3588"):
        rkllm_param = RKLLMParam()
        rkllm_param.model_path = bytes(model_path, 'utf-8')

        rkllm_param.max_context_len = 4096
        rkllm_param.max_new_tokens = 4096
        rkllm_param.skip_special_token = True
        rkllm_param.n_keep = -1
        rkllm_param.top_k = 1
        rkllm_param.top_p = 0.9
        rkllm_param.temperature = 0.8
        rkllm_param.repeat_penalty = 1.1
        rkllm_param.frequency_penalty = 0.0
        rkllm_param.presence_penalty = 0.0

        rkllm_param.mirostat = 0

```

```

rkllm_param.mirostat_tau = 5.0
rkllm_param.mirostat_eta = 0.1

rkllm_param.is_async = False

rkllm_param.ignore_eos_token = False

rkllm_param.extend_param.base_domain_id = 0
rkllm_param.extend_param.embed_flash = 1
rkllm_param.extend_param.n_batch = 1
rkllm_param.extend_param.use_cross_attn = 0
rkllm_param.extend_param.enabled_cpus_num = 4
if platform.lower() in ["rk3576", "rk3588"]:
    rkllm_param.extend_param.enabled_cpus_mask = (1 << 4) | (1 <<
5) | (1 << 6) | (1 << 7)
else:
    rkllm_param.extend_param.enabled_cpus_mask = (1 << 0) | (1 <<
1) | (1 << 2) | (1 << 3)

self.handle = RKLLM_Handle_t()

self.rkllm_init = rkllm_lib.rkllm_init
self.rkllm_init.argtypes = [ctypes.POINTER(RKLLM_Handle_t),
ctypes.POINTER(RKLLMParam), ctypes.POINTER(RKLLMCallback)]
self.rkllm_init.restype = ctypes.c_int
self.callback = RKLLMCallback()
self.callback.result_callback = callback
self.callback.result_userdata = None
self.callback.tokenizer_callback = LLMTokenizerCallback_type()
self.callback.tokenizer_userdata = None
self.callback.embed_callback = LLMGetEmbedCallback_type()
self.callback.embed_userdata = None
ret = self.rkllm_init(ctypes.byref(self.handle),
ctypes.byref(rkllm_param), ctypes.byref(self.callback))
if (ret != 0):
    print("\nrkllm init failed\n")
    exit(0)
else:
    print("\nrkllm init success!\n")

self.rkllm_run = rkllm_lib.rkllm_run

```



```

        self.rkllm_run.argtypes = [RKLLM_Handle_t,
ctypes.POINTER(RKLLMInput), ctypes.POINTER(RKLLMInferParam),
ctypes.c_void_p]
        self.rkllm_run.restype = ctypes.c_int

        self.set_chat_template = rkllm_lib.rkllm_set_chat_template
        self.set_chat_template.argtypes = [RKLLM_Handle_t,
ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p]
        self.set_chat_template.restype = ctypes.c_int

        self.set_function_tools_ = rkllm_lib.rkllm_set_function_tools
        self.set_function_tools_.argtypes = [RKLLM_Handle_t,
ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p]
        self.set_function_tools_.restype = ctypes.c_int

        self.rkllm_destroy = rkllm_lib.rkllm_destroy
        self.rkllm_destroy.argtypes = [RKLLM_Handle_t]
        self.rkllm_destroy.restype = ctypes.c_int

        self.rkllm_abort = rkllm_lib.rkllm_abort

        rkllm_lora_params = None
        if lora_model_path:
            lora_adapter_name = "test"
            lora_adapter = RKLLMLoraAdapter()
            ctypes.memset(ctypes.byref(lora_adapter), 0,
ctypes.sizeof(RKLLMLoraAdapter))
            lora_adapter.lora_adapter_path =
ctypes.c_char_p((lora_model_path).encode('utf-8'))
            lora_adapter.lora_adapter_name =
ctypes.c_char_p((lora_adapter_name).encode('utf-8'))
            lora_adapter.scale = 1.0

            rkllm_load_lora = rkllm_lib.rkllm_load_lora
            rkllm_load_lora.argtypes = [RKLLM_Handle_t,
ctypes.POINTER(RKLLMLoraAdapter)]
            rkllm_load_lora.restype = ctypes.c_int
            rkllm_load_lora(self.handle, ctypes.byref(lora_adapter))
            rkllm_lora_params = RKLLMLoraParam()
            rkllm_lora_params.lora_adapter_name =
ctypes.c_char_p((lora_adapter_name).encode('utf-8'))

        self.rkllm_infer_params = RKLLMInferParam()

```

```

        ctypes.memset(ctypes.byref(self.rkllm_infer_params), 0,
ctypes.sizeof(RKLLMInferParam))
        self.rkllm_infer_params.mode =
RKLLMInferMode.RKLLM_INFER_GENERATE
        self.rkllm_infer_params.lora_params =
ctypes.pointer(rkllm_lora_params) if rkllm_lora_params else None
        self.rkllm_infer_params.keep_history = 0

        self.prompt_cache_path = None
        if prompt_cache_path:
            self.prompt_cache_path = prompt_cache_path

            rkllm_load_prompt_cache = rkllm_lib.rkllm_load_prompt_cache
            rkllm_load_prompt_cache.argtypes = [RKLLM_Handle_t,
ctypes.c_char_p]
            rkllm_load_prompt_cache.restype = ctypes.c_int
            rkllm_load_prompt_cache(self.handle,
ctypes.c_char_p((prompt_cache_path).encode('utf-8')))

            self.tools = None
            self.model_name = os.path.basename(model_path).rsplit('.',
1)[0]

        def set_chat_template(self, system_prompt, prompt_prefix,
prompt_postfix):
            self.set_chat_template_(self.handle,
ctypes.c_char_p(system_prompt.encode('utf-8')),
ctypes.c_char_p(prompt_prefix.encode('utf-8')),
ctypes.c_char_p(prompt_postfix.encode('utf-8')))

        def set_function_tools(self, system_prompt, tools,
tool_response_str):
            if self.tools is None or not self.tools == tools:
                self.tools = tools
                self.set_function_tools_(self.handle,
ctypes.c_char_p(system_prompt.encode('utf-8')),
ctypes.c_char_p(tools.encode('utf-8')),
ctypes.c_char_p(tool_response_str.encode('utf-8')))

        def run(self, role, enable_thinking, prompt, sampling_params=None,
max_new_tokens=None):
            rkllm_input = RKLLMInput()

```

```

        rkllm_input.role = role.encode('utf-8') if role is not None
    else "user".encode('utf-8')

    rkllm_input.enable_thinking = ctypes.c_bool(enable_thinking if
enable_thinking is not None else False)

    rkllm_input.input_type = RKLLMInputType.RKLLM_INPUT_PROMPT
    rkllm_input.prompt_input =
ctypes.c_char_p(prompt.encode('utf-8'))

    # Apply per-request sampling params and max_new_tokens
    if sampling_params is not None:
        self.rkllm_infer_params.sampling_params =
ctypes.pointer(sampling_params)
    if max_new_tokens is not None:
        self.rkllm_infer_params.max_new_tokens = max_new_tokens

    self.rkllm_run(self.handle, ctypes.byref(rkllm_input),
ctypes.byref(self.rkllm_infer_params), None)

    # Reset sampling_params to NULL after run (avoid dangling
pointer)
    if sampling_params is not None:
        self.rkllm_infer_params.sampling_params = None
    # Reset max_new_tokens to 0 after run (<=0 means use init
value)
    if max_new_tokens is not None:
        self.rkllm_infer_params.max_new_tokens = 0
    return

def abort(self):
    return self.rkllm_abort(self.handle)

def release(self):
    self.rkllm_destroy(self.handle)

def extract_system_prompt_and_tools(messages):
    """Extract system prompt and tools from messages."""
    system_prompt = ""
    tools = None
    for msg in messages:
        if msg.get("role") == "system":
            system_prompt = msg.get("content", "")

```

```

    return system_prompt, tools

# Track previous messages to detect newly added ones across requests
_last_messages = []

def get_last_input(messages):
    """Extract the latest new input for inference.

    Tracks message history to detect newly added messages since last
    call.

    When multiple consecutive 'tool' messages are detected in the new
    batch,
    merges their content into a JSON array, e.g.:
        [{"temperature": 26.1, ...}, {"temperature": 25.9, ...}]

    Otherwise returns the last user/tool message's role and content
    directly.
    """
    global _last_messages

    role = "user"
    content = ""

    # Detect newly added messages by comparing with tracked history
    prev_len = len(_last_messages)
    new_messages = messages[prev_len:] if prev_len < len(messages) else
[]

    # Update tracking for next call
    _last_messages = list(messages)

    if not new_messages:
        # No new messages - fall back to last user/tool in full history
        for msg in reversed(messages):
            r = msg.get("role", "")
            if r in ("user", "tool"):
                role = r
                content = _extract_text_content(msg)
                break
        return role, content

```

```

    # Filter new messages to only user/tool roles
    new_inputs = [m for m in new_messages if m.get("role", "") in
("user", "tool")]

    if not new_inputs:
        # New messages exist but none are user/tool (e.g. only
assistant)

        for msg in reversed(messages):
            r = msg.get("role", "")
            if r in ("user", "tool"):
                role = r
                content = _extract_text_content(msg)
                break
        return role, content

    # Check if all new inputs are 'tool' role → merge into JSON array
    if all(m.get("role") == "tool" for m in new_inputs):
        tool_contents = []
        for m in new_inputs:
            c = m.get("content", "")
            try:
                parsed = json.loads(c)
                tool_contents.append(parsed)
            except (json.JSONDecodeError, TypeError):
                tool_contents.append(c)
        role = "tool"
        content = json.dumps(tool_contents, ensure_ascii=False)
    else:
        # Use the last new user/tool message
        last = new_inputs[-1]
        role = last.get("role", "user")
        content = _extract_text_content(last)

    return role, content

def _extract_text_content(msg):
    """Extract plain text content from a message dict."""
    c = msg.get("content", "")
    if isinstance(c, list):
        text_parts = [p.get("text", "") for p in c if p.get("type") ==
"text"]
    return " ".join(text_parts)

```

```

        return c or ""

def run_inference(rkllm_model, prompt, role="user",
enable_thinking=False, sampling_params=None, max_new_tokens=None):
    """Run inference in a thread and wait for completion. Returns
output text."""
    global global_text, global_state
    global_text = []
    global_state = -1

    model_thread = threading.Thread(target=rkllm_model.run, args=(role,
enable_thinking, prompt, sampling_params, max_new_tokens))
    model_thread.start()

    output = ""
    model_thread_finished = False
    while not model_thread_finished:
        while len(global_text) > 0:
            output += global_text.pop(0)
            time.sleep(0.005)
        model_thread.join(timeout=0.005)
        model_thread_finished = not model_thread.is_alive()
    return output

if __name__ == "__main__":

    # Set resource limit
    resource.setrlimit(resource.RLIMIT_NOFILE, (102400, 102400))

    # Initialize RKLLM model
    print("=====init....=====")
    sys.stdout.flush()
    rkllm_model = RKLLM("/mnt/nvme01/rkllm_model/qwen3_5_4b.rkllm", "",
"", "rk3588")
    print(f"Model name: {rkllm_model.model_name}")
    print("=====")
    sys.stdout.flush()

    # Graceful shutdown on Ctrl+C
    def shutdown_handler(signum, frame):

```

```

        print("\n=====")
        print("Received interrupt signal, releasing RKLLM model
resources...")
        rkllm_model.release()
        print("=====")
        sys.exit(0)

# 8. Interactive loop
try:
    while True:
        user_input = input("\nYou: ")
        if user_input.lower() in ['exit', 'quit']:
            break
        if not user_input.strip():
            continue

        print("\nQwen: ", end="", flush=True)

        data = {
            "model": "rkllm",                      # Model identifier
(customizable)
            "messages": [
                {"role": "user", "content": user_input}
            ],
            "stream": False,                      # True =
streaming, False = non-streaming
            "temperature": 0.8,                  # Sampling
temperature (default: 0.8)
            "top_p": 0.9,                      # Nucleus sampling
threshold (default: 0.9)
            "top_k": 1,                        # Top-k sampling
(default: 1)
            "max_tokens": 4096,                # Max tokens to
generate (default: 4096)
            "repeat_penalty": 1.1,             # Repeat penalty
(default: 1.1)
            "frequency_penalty": 0.0,          # Frequency
penalty (default: 0.0)
            "presence_penalty": 0.0,           # Presence penalty
(default: 0.0)
            "enable_thinking": False,          # Enable deep
thinking mode
        }

```

```

messages = data["messages"]
model_name = data.get("model", rkllm_model.model_name)
stream = data.get("stream", False)
temperature = data.get("temperature", 0.8)
top_p = data.get("top_p", 0.9)
top_k = data.get("top_k", 1)
max_tokens = data.get("max_tokens", 4096)
repeat_penalty = data.get("repeat_penalty", 1.1)
frequency_penalty = data.get("frequency_penalty", 0.0)
presence_penalty = data.get("presence_penalty", 0.0)
enable_thinking = data.get("enable_thinking", False)
tools = data.get("tools", None)

# Build per-request sampling params
sampling_params = RKLLMSamplingParam()
sampling_params.top_k = int(top_k)
sampling_params.top_p = float(top_p)
sampling_params.temperature = float(temperature)
sampling_params.repeat_penalty = float(repeat_penalty)
sampling_params.frequency_penalty =
float(frequency_penalty)
sampling_params.presence_penalty = float(presence_penalty)
sampling_params.mirostat = 0
sampling_params.mirostat_tau = 5.0
sampling_params.mirostat_eta = 0.1

# Extract system prompt
sys_prompt = ""
for msg in messages:
    if msg.get("role") == "system":
        sys_prompt = msg.get("content", "")

# Configure function calling if tools are provided
if tools:
    rkllm_model.set_function_tools(
        system_prompt=sys_prompt,
        tools=json.dumps(tools),
        tool_response_str="tool_response"
    )
role, prompt = get_last_input(messages)
output = run_inference(rkllm_model, prompt, role,
enable_thinking, sampling_params, max_tokens)

```



```

        print(output)

    finally:
        signal.signal(signal.SIGINT, shutdown_handler)
        signal.signal(signal.SIGTERM, shutdown_handler)

```

=====

Sample code 2: It shows qwen3.5 model does not cache saving.

```

# CMD="python3 flask_server.py --rkllm_model_path $MODEL_PATH
--target_platform $TARGET_PLATFORM --lora_model_path $LORA_PATH
--prompt_cache_path $PROMPT_FILE_PATH"
import ctypes
import sys
import os
import subprocess
import resource
import threading
import time
import argparse
import json
import uuid
import signal

# Set the dynamic library path
rkllm_lib = ctypes.CDLL('/usr/lib/librkllmrt.so')
# Define the structures from the library
RKLLM_Handle_t = ctypes.c_void_p
userdata = ctypes.c_void_p(None)

LLMCallState = ctypes.c_int
LLMCallState.RKLLM_RUN_NORMAL = 0
LLMCallState.RKLLM_RUN_WAITING = 1
LLMCallState.RKLLM_RUN_FINISH = 2
LLMCallState.RKLLM_RUN_ERROR = 3

RKLLMInputType = ctypes.c_int
RKLLMInputType.RKLLM_INPUT_PROMPT = 0
RKLLMInputType.RKLLM_INPUT_TOKEN = 1
RKLLMInputType.RKLLM_INPUT_EMBED = 2

```

```

RKLLMInputType.RKLLM_INPUT_MULTIMODAL = 3

RKLLMInferMode = ctypes.c_int
RKLLMInferMode.RKLLM_INFER_GENERATE = 0
RKLLMInferMode.RKLLM_INFER_GET_LAST_HIDDEN_LAYER = 1
RKLLMInferMode.RKLLM_INFER_GET_LOGITS = 2
class RKLLMExtendParam(ctypes.Structure):
    _fields_ = [
        ("base_domain_id", ctypes.c_int32),
        ("embed_flash", ctypes.c_int8),
        ("enabled_cpus_num", ctypes.c_int8),
        ("enabled_cpus_mask", ctypes.c_uint32),
        ("n_batch", ctypes.c_uint8),
        ("use_cross_attn", ctypes.c_int8),
        ("reserved", ctypes.c_uint8 * 104)
    ]

class RKLLMParam(ctypes.Structure):
    _fields_ = [
        ("model_path", ctypes.c_char_p),
        ("max_context_len", ctypes.c_int32),
        ("max_new_tokens", ctypes.c_int32),
        ("top_k", ctypes.c_int32),
        ("n_keep", ctypes.c_int32),
        ("top_p", ctypes.c_float),
        ("temperature", ctypes.c_float),
        ("repeat_penalty", ctypes.c_float),
        ("frequency_penalty", ctypes.c_float),
        ("presence_penalty", ctypes.c_float),
        ("mirostat", ctypes.c_int32),
        ("mirostat_tau", ctypes.c_float),
        ("mirostat_eta", ctypes.c_float),
        ("skip_special_token", ctypes.c_bool),
        ("ignore_eos_token", ctypes.c_bool),
        ("is_async", ctypes.c_bool),
        ("extend_param", RKLLMExtendParam),
    ]

class RKLLMLoraAdapter(ctypes.Structure):
    _fields_ = [
        ("lora_adapter_path", ctypes.c_char_p),
        ("lora_adapter_name", ctypes.c_char_p),
        ("scale", ctypes.c_float)
    ]

```

```

]

class RKLLMEmbedInput(ctypes.Structure):
    _fields_ = [
        ("embed", ctypes.POINTER(ctypes.c_float)),
        ("n_tokens", ctypes.c_size_t)
    ]

class RKLLMTokenInput(ctypes.Structure):
    _fields_ = [
        ("input_ids", ctypes.POINTER(ctypes.c_int32)),
        ("n_tokens", ctypes.c_size_t)
    ]

class RKLLMImageInput(ctypes.Structure):
    _fields_ = [
        ("image_embed", ctypes.POINTER(ctypes.c_float)),
        ("n_image_tokens", ctypes.c_size_t),
        ("n_image", ctypes.c_size_t),
        ("image_start", ctypes.c_char_p),
        ("image_end", ctypes.c_char_p),
        ("image_content", ctypes.c_char_p),
        ("image_width", ctypes.c_size_t),
        ("image_height", ctypes.c_size_t),
    ]

class RKLLMVideoInput(ctypes.Structure):
    _fields_ = [
        ("video_embed", ctypes.POINTER(ctypes.c_float)),
        ("n_frame_tokens", ctypes.c_size_t),
        ("n_frame_per_video", ctypes.c_size_t),
        ("n_video", ctypes.c_size_t),
        ("video_start", ctypes.c_char_p),
        ("video_end", ctypes.c_char_p),
        ("video_content", ctypes.c_char_p),
        ("frame_width", ctypes.c_size_t),
        ("frame_height", ctypes.c_size_t),
    ]

class RKLLMMultiModalInput(ctypes.Structure):
    _fields_ = [
        ("prompt", ctypes.c_char_p),
        ("image", RKLLMImageInput),
    ]

```

```

        ("video", RKLLMVideoInput),
    ]

class RKLLMInputUnion(ctypes.Union):
    _fields_ = [
        ("prompt_input", ctypes.c_char_p),
        ("embed_input", RKLLMEmbedInput),
        ("token_input", RKLLMTokenInput),
        ("multimodal_input", RKLLMMultiModalInput)
    ]

class RKLLMInput(ctypes.Structure):
    _anonymous_ = ("input_data",)
    _fields_ = [
        ("role", ctypes.c_char_p),
        ("enable_thinking", ctypes.c_bool),
        ("input_type", RKLLMInputType),
        ("input_data", RKLLMInputUnion)
    ]

class RKLLMLoraParam(ctypes.Structure):
    _fields_ = [
        ("lora_adapter_name", ctypes.c_char_p)
    ]

class RKLLMPromptCacheParam(ctypes.Structure):
    _fields_ = [
        ("save_prompt_cache", ctypes.c_int),
        ("prompt_cache_path", ctypes.c_char_p)
    ]

class RKLLMSamplingParam(ctypes.Structure):
    _fields_ = [
        ("top_k", ctypes.c_int32),
        ("top_p", ctypes.c_float),
        ("temperature", ctypes.c_float),
        ("repeat_penalty", ctypes.c_float),
        ("frequency_penalty", ctypes.c_float),
        ("presence_penalty", ctypes.c_float),
        ("mirostat", ctypes.c_int32),
        ("mirostat_tau", ctypes.c_float),
        ("mirostat_eta", ctypes.c_float),
    ]

```

```
class RKLLMInferParam(ctypes.Structure):
    _fields_ = [
        ("mode", RKLLMInferMode),
        ("lora_params", ctypes.POINTER(RKLLMLoraParam)),
        ("prompt_cache_params", ctypes.POINTER(RKLLMPromptCacheParam)),
        ("sampling_params", ctypes.POINTER(RKLLMSamplingParam)),
        ("keep_history", ctypes.c_int),
        ("max_new_tokens", ctypes.c_int32),
    ]

class RKLLMResultLastHiddenLayer(ctypes.Structure):
    _fields_ = [
        ("hidden_states", ctypes.POINTER(ctypes.c_float)),
        ("embd_size", ctypes.c_int),
        ("num_tokens", ctypes.c_int)
    ]

class RKLLMResultLogits(ctypes.Structure):
    _fields_ = [
        ("logits", ctypes.POINTER(ctypes.c_float)),
        ("vocab_size", ctypes.c_int),
        ("num_tokens", ctypes.c_int)
    ]

class RKLLMPerfStat(ctypes.Structure):
    _fields_ = [
        ("prefill_time_ms", ctypes.c_float),
        ("prefill_tokens", ctypes.c_int),
        ("generate_time_ms", ctypes.c_float),
        ("generate_tokens", ctypes.c_int),
        ("memory_usage_mb", ctypes.c_float)
    ]

class RKLLMResult(ctypes.Structure):
    _fields_ = [
        ("text", ctypes.c_char_p),
        ("token_id", ctypes.c_int),
        ("last_hidden_layer", RKLLMResultLastHiddenLayer),
        ("logits", RKLLMResultLogits),
        ("perf", RKLLMPerfStat)
    ]
```

```

# Define global variables to store the callback function output
system_prompt = ''
global_text = []
global_state = -1
split_byte_data = bytes(b'') # Used to store the segmented byte data

# Per-request context for function calling
tool_call_context = {"pending_tools": None, "tool_response_str":
"tool_response"}

# Define the callback function
def callback_impl(result, userdata, state):
    global global_text, global_state, split_byte_data
    if state == LLMCallState.RKLLM_RUN_FINISH:
        global_state = state
        sys.stdout.flush()
    elif state == LLMCallState.RKLLM_RUN_ERROR:
        global_state = state
        print("run error")
        sys.stdout.flush()
    elif state == LLMCallState.RKLLM_RUN_NORMAL:
        global_state = state
        global_text += result.contents.text.decode('utf-8')
    return 0

# Connect the callback function between the Python side and the C++
side
LLMResultCallback_type = ctypes.CFUNCTYPE(ctypes.c_int,
ctypes.POINTER(RKLLMResult), ctypes.c_void_p, ctypes.c_int)
LLMTokenizerCallback_type = ctypes.CFUNCTYPE(ctypes.c_int,
ctypes.c_void_p, ctypes.c_char_p, ctypes.c_int32,
ctypes.POINTER(ctypes.c_int32), ctypes.c_int32)
LLMGetEmbedCallback_type = ctypes.CFUNCTYPE(ctypes.c_int,
ctypes.c_void_p, ctypes.POINTER(ctypes.c_int32), ctypes.c_uint64,
ctypes.c_void_p, ctypes.c_uint64)

class RKLLMCallback(ctypes.Structure):
    _fields_ = [
        ("result_callback", LLMResultCallback_type),
        ("result_userdata", ctypes.c_void_p),
        ("tokenizer_callback", LLMTokenizerCallback_type),

```

```

        ("tokenizer_userdata", ctypes.c_void_p),
        ("embed_callback", LLMGetEmbedCallback_type),
        ("embed_userdata", ctypes.c_void_p),
    ]

callback = LLMResultCallback_type(callback_impl)

# Define the RKLLM class, which includes initialization, inference, and
# release operations for the RKLLM model in the dynamic library
class RKLLM(object):
    def __init__(self, model_path, lora_model_path = None,
prompt_cache_path = None, platform = "rk3588"):
        rkllm_param = RKLLMParam()
        rkllm_param.model_path = bytes(model_path, 'utf-8')

        rkllm_param.max_context_len = 4096
        rkllm_param.max_new_tokens = 4096
        rkllm_param.skip_special_token = True
        rkllm_param.n_keep = -1
        rkllm_param.top_k = 1
        rkllm_param.top_p = 0.9
        rkllm_param.temperature = 0.8
        rkllm_param.repeat_penalty = 1.1
        rkllm_param.frequency_penalty = 0.0
        rkllm_param.presence_penalty = 0.0

        rkllm_param.mirostat = 0
        rkllm_param.mirostat_tau = 5.0
        rkllm_param.mirostat_eta = 0.1

        rkllm_param.is_async = False

        rkllm_param.ignore_eos_token = False

        rkllm_param.extend_param.base_domain_id = 0
        rkllm_param.extend_param.embed_flash = 1
        rkllm_param.extend_param.n_batch = 1
        rkllm_param.extend_param.use_cross_attn = 0
        rkllm_param.extend_param.enabled_cpus_num = 4
        if platform.lower() in ["rk3576", "rk3588"]:
            rkllm_param.extend_param.enabled_cpus_mask = (1 << 4) | (1 <<
5) | (1 << 6) | (1 << 7)
        else:

```

```

        rkllm_param.extend_param.enabled_cpus_mask = (1 << 0) | (1 <<
1) | (1 << 2) | (1 << 3)

    self.handle = RKLLM_Handle_t()

    self.rkllm_init = rkllm_lib.rkllm_init
    self.rkllm_init.argtypes = [ctypes.POINTER(RKLLM_Handle_t),
ctypes.POINTER(RKLLMParam), ctypes.POINTER(RKLLMCallback)]
    self.rkllm_init.restype = ctypes.c_int
    self.callback = RKLLMCallback()
    self.callback.result_callback = callback
    self.callback.result_userdata = None
    self.callback.tokenizer_callback = LLMTokenizerCallback_type()
    self.callback.tokenizer_userdata = None
    self.callback.embed_callback = LLMGetEmbedCallback_type()
    self.callback.embed_userdata = None
    ret = self.rkllm_init(ctypes.byref(self.handle),
ctypes.byref(rkllm_param), ctypes.byref(self.callback))
    if (ret != 0):
        print("\nrkllm init failed\n")
        exit(0)
    else:
        print("\nrkllm init success!\n")

    self.rkllm_run = rkllm_lib.rkllm_run
    self.rkllm_run.argtypes = [RKLLM_Handle_t,
ctypes.POINTER(RKLLMInput), ctypes.POINTER(RKLLMInferParam),
ctypes.c_void_p]
    self.rkllm_run.restype = ctypes.c_int

    self.set_chat_template = rkllm_lib.rkllm_set_chat_template
    self.set_chat_template.argtypes = [RKLLM_Handle_t,
ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p]
    self.set_chat_template.restype = ctypes.c_int

    self.set_function_tools_ = rkllm_lib.rkllm_set_function_tools
    self.set_function_tools_.argtypes = [RKLLM_Handle_t,
ctypes.c_char_p, ctypes.c_char_p, ctypes.c_char_p]
    self.set_function_tools_.restype = ctypes.c_int

    self.rkllm_destroy = rkllm_lib.rkllm_destroy
    self.rkllm_destroy.argtypes = [RKLLM_Handle_t]
    self.rkllm_destroy.restype = ctypes.c_int

```



```

self.rkllm_abort = rkllm_lib.rkllm_abort

rkllm_lora_params = None
if lora_model_path:
    lora_adapter_name = "test"
    lora_adapter = RKLLMLoraAdapter()
    ctypes.memset(ctypes.byref(lora_adapter), 0,
ctypes.sizeof(RKLLMLoraAdapter))
    lora_adapter.lora_adapter_path =
ctypes.c_char_p((lora_model_path).encode('utf-8'))
    lora_adapter.lora_adapter_name =
ctypes.c_char_p((lora_adapter_name).encode('utf-8'))
    lora_adapter.scale = 1.0

    rkllm_load_lora = rkllm_lib.rkllm_load_lora
    rkllm_load_lora.argtypes = [RKLLM_Handle_t,
ctypes.POINTER(RKLLMLoraAdapter)]
    rkllm_load_lora.restype = ctypes.c_int
    rkllm_load_lora(self.handle, ctypes.byref(lora_adapter))
    rkllm_lora_params = RKLLMLoraParam()
    rkllm_lora_params.lora_adapter_name =
ctypes.c_char_p((lora_adapter_name).encode('utf-8'))

self.rkllm_infer_params = RKLLMInferParam()
ctypes.memset(ctypes.byref(self.rkllm_infer_params), 0,
ctypes.sizeof(RKLLMInferParam))
self.rkllm_infer_params.mode =
RKLLMInferMode.RKLLM_INFER_GENERATE
self.rkllm_infer_params.lora_params =
ctypes.pointer(rkllm_lora_params) if rkllm_lora_params else None
self.rkllm_infer_params.keep_history = 1

self.prompt_cache_path = None
if prompt_cache_path:
    self.prompt_cache_path = prompt_cache_path

    if os.stat(self.prompt_cache_path).st_size != 0:
        rkllm_load_prompt_cache =
rkllm_lib.rkllm_load_prompt_cache
        rkllm_load_prompt_cache.argtypes = [RKLLM_Handle_t,
ctypes.c_char_p]
        rkllm_load_prompt_cache.restype = ctypes.c_int

```

```

        rkllm_load_prompt_cache(self.handle,
cypes.c_char_p((prompt_cache_path).encode('utf-8')))

        # Define the exact destination where the NPU will
auto-flush the binary matrix file
        cache_config = RKLLMPromptCacheParam(
            save_prompt_cache=1, # Enables active writing loops
            prompt_cache_path=(prompt_cache_path).encode('utf-8')
        )

self.rkllm_infer_params.prompt_cache_params=cypes.pointer(cache_config
)

        self.tools = None
        self.model_name = os.path.basename(model_path).rsplit('.',
1)[0]

        def set_chat_template(self, system_prompt, prompt_prefix,
prompt_postfix):
            self.set_chat_template_(self.handle,
cypes.c_char_p(system_prompt.encode('utf-8')),
cypes.c_char_p(prompt_prefix.encode('utf-8')),
cypes.c_char_p(prompt_postfix.encode('utf-8')))

        def set_function_tools(self, system_prompt, tools,
tool_response_str):
            if self.tools is None or not self.tools == tools:
                self.tools = tools
                self.set_function_tools_(self.handle,
cypes.c_char_p(system_prompt.encode('utf-8')),
cypes.c_char_p(tools.encode('utf-8')),
cypes.c_char_p(tool_response_str.encode('utf-8')))

        def run(self, role, enable_thinking, prompt, sampling_params=None,
max_new_tokens=None):
            rkllm_input = RKLLMInput()
            rkllm_input.role = role.encode('utf-8') if role is not None
else "user".encode('utf-8')
            rkllm_input.enable_thinking = cypes.c_bool(enable_thinking if
enable_thinking is not None else False)
            rkllm_input.input_type = RKLLMInputType.RKLLM_INPUT_PROMPT
            rkllm_input.prompt_input =
cypes.c_char_p(prompt.encode('utf-8'))

```

```

        # Apply per-request sampling params and max_new_tokens
        if sampling_params is not None:
            self.rkllm_infer_params.sampling_params =
ctypes.pointer(sampling_params)
        if max_new_tokens is not None:
            self.rkllm_infer_params.max_new_tokens = max_new_tokens

        self.rkllm_run(self.handle, ctypes.byref(rkllm_input),
ctypes.byref(self.rkllm_infer_params), None)

        # Reset sampling_params to NULL after run (avoid dangling
pointer)
        if sampling_params is not None:
            self.rkllm_infer_params.sampling_params = None
        # Reset max_new_tokens to 0 after run (<=0 means use init
value)
        if max_new_tokens is not None:
            self.rkllm_infer_params.max_new_tokens = 0
        return

def abort(self):
    return self.rkllm_abort(self.handle)

def release(self):
    self.rkllm_destroy(self.handle)

def extract_system_prompt_and_tools(messages):
    """Extract system prompt and tools from messages."""
    system_prompt = ""
    tools = None
    for msg in messages:
        if msg.get("role") == "system":
            system_prompt = msg.get("content", "")
    return system_prompt, tools

# Track previous messages to detect newly added ones across requests
_last_messages = []

```

```

def get_last_input(messages):
    """Extract the latest new input for inference.

    Tracks message history to detect newly added messages since last
    call.

    When multiple consecutive 'tool' messages are detected in the new
    batch,
    merges their content into a JSON array, e.g.:
        [{"temperature": 26.1, ...}, {"temperature": 25.9, ...}]

    Otherwise returns the last user/tool message's role and content
    directly.
    """
    global _last_messages

    role = "user"
    content = ""

    # Detect newly added messages by comparing with tracked history
    prev_len = len(_last_messages)
    new_messages = messages[prev_len:] if prev_len < len(messages) else
[]

    # Update tracking for next call
    _last_messages = list(messages)

    if not new_messages:
        # No new messages - fall back to last user/tool in full history
        for msg in reversed(messages):
            r = msg.get("role", "")
            if r in ("user", "tool"):
                role = r
                content = _extract_text_content(msg)
                break
        return role, content

    # Filter new messages to only user/tool roles
    new_inputs = [m for m in new_messages if m.get("role", "") in
("user", "tool")]

    if not new_inputs:
        # New messages exist but none are user/tool (e.g. only
assistant)

```

```

        for msg in reversed(messages):
            r = msg.get("role", "")
            if r in ("user", "tool"):
                role = r
                content = _extract_text_content(msg)
                break
        return role, content

# Check if all new inputs are 'tool' role → merge into JSON array
if all(m.get("role") == "tool" for m in new_inputs):
    tool_contents = []
    for m in new_inputs:
        c = m.get("content", "")
        try:
            parsed = json.loads(c)
            tool_contents.append(parsed)
        except (json.JSONDecodeError, TypeError):
            tool_contents.append(c)
    role = "tool"
    content = json.dumps(tool_contents, ensure_ascii=False)
else:
    # Use the last new user/tool message
    last = new_inputs[-1]
    role = last.get("role", "user")
    content = _extract_text_content(last)

return role, content


def _extract_text_content(msg):
    """Extract plain text content from a message dict."""
    c = msg.get("content", "")
    if isinstance(c, list):
        text_parts = [p.get("text", "") for p in c if p.get("type") == "text"]
        return " ".join(text_parts)
    return c or ""


def run_inference(rkllm_model, prompt, role="user",
enable_thinking=False, sampling_params=None, max_new_tokens=None):
    """Run inference in a thread and wait for completion. Returns
output text."""

```

```

global global_text, global_state
global_text = []
global_state = -1

model_thread = threading.Thread(target=rkllm_model.run, args=(role,
enable_thinking, prompt, sampling_params, max_new_tokens))
model_thread.start()

output = ""
model_thread_finished = False
while not model_thread_finished:
    while len(global_text) > 0:
        output += global_text.pop(0)
        time.sleep(0.005)
    model_thread.join(timeout=0.005)
    model_thread_finished = not model_thread.is_alive()
return output

if __name__ == "__main__":

    # Set resource limit
    resource.setrlimit(resource.RLIMIT_NOFILE, (102400, 102400))

    # Initialize RKLLM model
    print("=====init....=====")
    sys.stdout.flush()
    rkllm_model = RKLLM("/mnt/nvme01/rkllm_model/qwen3_5_4b.rkllm", "",
"/mnt/nvme01/rkllm_model/prompt_kv.cache", "rk3588")
    print(f"Model name: {rkllm_model.model_name}")
    print("=====")
    sys.stdout.flush()

    # Graceful shutdown on Ctrl+C
    def shutdown_handler(signum, frame):
        print("\n=====")
        print("Received interrupt signal, releasing RKLLM model
resources...")
        rkllm_model.release()
        print("=====")
        sys.exit(0)

```

```

# 8. Interactive loop
try:
    while True:
        user_input = input("\nYou: ")
        if user_input.lower() in ['exit', 'quit']:
            break
        if not user_input.strip():
            continue

        print("\nQwen: ", end="", flush=True)

        data = {
            "model": "rkllm",          # Model identifier
(customizable)
            "messages": [
                {"role": "user", "content": user_input}
            ],
            "stream": False,          # True =
streaming, False = non-streaming
            "temperature": 0.8,      # Sampling
temperature (default: 0.8)
            "top_p": 0.9,            # Nucleus sampling
threshold (default: 0.9)
            "top_k": 1,              # Top-k sampling
(default: 1)
            "max_tokens": 4096,      # Max tokens to
generate (default: 4096)
            "repeat_penalty": 1.1,   # Repeat penalty
(default: 1.1)
            "frequency_penalty": 0.0, # Frequency
penalty (default: 0.0)
            "presence_penalty": 0.0, # Presence penalty
(default: 0.0)
            "enable_thinking": False, # Enable deep
thinking mode
        }

        messages = data["messages"]
        model_name = data.get("model", rkllm_model.model_name)
        stream = data.get("stream", False)
        temperature = data.get("temperature", 0.8)
        top_p = data.get("top_p", 0.9)
        top_k = data.get("top_k", 1)

```

```

max_tokens = data.get("max_tokens", 4096)
repeat_penalty = data.get("repeat_penalty", 1.1)
frequency_penalty = data.get("frequency_penalty", 0.0)
presence_penalty = data.get("presence_penalty", 0.0)
enable_thinking = data.get("enable_thinking", False)
tools = data.get("tools", None)

# Build per-request sampling params
sampling_params = RKLLMSamplingParam()
sampling_params.top_k = int(top_k)
sampling_params.top_p = float(top_p)
sampling_params.temperature = float(temperature)
sampling_params.repeat_penalty = float(repeat_penalty)
sampling_params.frequency_penalty =
float(frequency_penalty)
sampling_params.presence_penalty = float(presence_penalty)
sampling_params.mirostat = 0
sampling_params.mirostat_tau = 5.0
sampling_params.mirostat_eta = 0.1

# Extract system prompt
sys_prompt = ""
for msg in messages:
    if msg.get("role") == "system":
        sys_prompt = msg.get("content", "")

# Configure function calling if tools are provided
if tools:
    rkllm_model.set_function_tools(
        system_prompt=sys_prompt,
        tools=json.dumps(tools),
        tool_response_str="tool_response"
    )
    role, prompt = get_last_input(messages)
    output = run_inference(rkllm_model, prompt, role,
enable_thinking, sampling_params, max_tokens)
    print(output)

finally:
    signal.signal(signal.SIGINT, shutdown_handler)
    signal.signal(signal.SIGTERM, shutdown_handler)

```


=====

Performance fix/shell crash

=====

use 100 watt adapter to power board.

ubuntu@ubuntu:~\$ export RKNPU_TIMEOUT=60

ubuntu@ubuntu:~\$ sudo ./fix_freq_rklm.sh

NPU available frequencies:

300000000 400000000 500000000 600000000 700000000 800000000 900000000
1000000000

Fix NPU max frequency:

1000000000

CPU available frequencies:

408000 600000 816000 1008000 1200000 1416000 1608000 1800000
408000 600000 816000 1008000 1200000 1416000 1608000 1800000 2016000 2208000
2256000
408000 600000 816000 1008000 1200000 1416000 1608000 1800000 2016000 2208000
2304000

Fix CPU max frequency:

1800000

2256000

2304000

GPU available frequencies:

1000000000 900000000 800000000 700000000 600000000 500000000 400000000
300000000

Fix GPU max frequency:

1000000000

DDR available frequencies:

534000000 1320000000 1968000000 2400000000

Fix DDR max frequency:

2400000000

ubuntu@ubuntu:~\$